WOD2Sim: Contract-Based System Integration of Dataset-Trained Driving Policies into Distributed Closed-Loop Simulation

Alba Maria Tellez Fernandez
Independent Researcher
Email: amtellezfernandez@gmail.com

***Abstract*—Offline driving policies are usually trained around logged observations, route context, and short-horizon trajectory targets, while distributed closed-loop simulators require persistent services, runtime clocks, deployment provenance, and auditable evidence. WOD2Sim treats this mismatch as a contract-based system-integration problem rather than as a new driving-policy benchmark. It defines five contracts for executing dataset-trained trajectory policies as AlpaSim external drivers: semantic route and scene preservation, temporal trajectory adaptation, explicit lifecycle handling, deployment provenance, and evidence validity. The implementation provides dependency-light baselines, optional learned and actor-aware entry points, deterministic launch manifests, route/sensor audits, aggregate reports, and generated paper artifacts. In the current contract-validation matrix, the public dependency-light core completes 30/30 closed-loop rows over 15 local scenes, and 42/45 completed full-contract rollouts pass route and sensor audits. The evidence gate falsely blocks 0/ 42 valid full-contract rows. Against a runnable command-only route wrapper, matched semantic ablations complete 15/15 metric-bearing pairs: a non-contract path could accept 15/15 invalid route rows as policy evidence, while WOD2Sim rejects 15/15 as non-claim-valid. These results support a bounded conclusion: explicit integration contracts make valid rollouts executable and prevent integration-invalid evidence from being misreported as policy behavior. The paper does not claim learned-policy superiority, scenario-category coverage, or public benchmark validity.**

I. Introduction

Dataset-trained driving policies and closed-loop simulators expose different failure surfaces. A WOD-style policy consumes logged observations, route intent or route geometry, and a short-horizon ego-relative trajectory target. A closed-loop external driver must instead remain alive as a service, accept incremental sensor and route messages, return controls on the simulator clock, and leave an evidence trail that distinguishes policy behavior from integration failure.

This distinction matters because a wrapper can make integration failures look like policy failures. For example, reducing route geometry to a high-level command can remove the local polyline used by a route-conditioned trajectory policy. A later collision, route drift, or invalid trajectory may then be attributed to policy quality even though the decision-relevant input was lost at the simulator boundary. Similar confounds arise from mismatched trajectory sampling grids, stale observations, duplicate close messages, optional backend imports, missing scene assets, and audit artifacts that cannot support scientific claims.

WOD2Sim addresses this problem as system integration. It provides a reference boundary for running heterogeneous trajectory policies through AlpaSim's external-driver interface while making each boundary assumption inspectable. The effectiveness gain is not a higher driving score; it is earlier and auditable attribution of integration defects before they are misread as policy behavior. The central claim is therefore not that WOD2Sim is a new driving policy or an official WOD benchmark implementation. The claim is that dataset-trained policies require explicit semantic, temporal, lifecycle, deployment, and evidence contracts before closed-loop results can be interpreted.

This paper asks two integration-effectiveness questions: which contract mismatches prevent dataset-trained policies from operating as reliable simulator services, and can explicit contract validation distinguish integration and runtime failures from policy behavior failures? The current repository deliberately separates the public release core from gated extensions. The core release exercises dependency-light adapters, route-preservation audits, evidence gates, and semantic ablations without requiring learned checkpoints, actor proxies, or redistributed restricted scenes. Direct actor-aware rows, learned-checkpoint execution, and complete benchmark publication remain gated extension work. The evidence package keeps those blockers traceable without making them dependencies of the core integration claim.

A. Failure Attribution Rule

WOD2Sim uses an explicit attribution rule before reporting any behavior or failure metric. A rollout event may be interpreted as policy behavior or policy failure only when the semantic route contract, temporal adapter, lifecycle state, deployment preconditions, and evidence audit all pass. If any of those gates fails, the row is classified as an integration, precondition, or evidence failure. A missing actor proxy is therefore not a weak planner, a command-proxy route is not a bad route-conditioned policy, and an incomplete manifest is not a collision result. Passing all gates permits policy-behavior attribution, not automatic policy-failure attribution; policy failure additionally requires the retained failure layer to be policy. This rule is intentionally stricter than ordinary demo reporting because the scientific risk is misattributing adapter or runtime faults to the policy being integrated. WOD2Sim

TABLE I
CONTRACT MAP FOR INTEGRATING WOD-STYLE TRAJECTORY POLICIES INTO DISTRIBUTED CLOSED-LOOP SIMULATION.

Mismatch	Contract	Mechanism	Validation
Command-only route	Semantic	Preserve route geometry	route-source audit
Policy horizon/runtime grid	Temporal	Deterministic resampling	cadence tests
Script flow/session service	Lifecycle	Idempotent late-event handling	lifecycle tests
Implicit host state	Deployment	Materialized manifests	readiness checks
Process exit/evidence	Evidence	Audit-valid summaries	claim gate

therefore evaluates integration validity first and policy quality only after the row is claim-valid.

II. INTEGRATION PROBLEM AND REQUIREMENTS

The offline-policy contract and the external-driver contract differ along five layers. Table I summarizes the recurring mismatches and the WOD2Sim mechanisms used to expose them.

A. Semantic Contract

The semantic contract preserves decision-relevant meaning across the dataset-policy and simulator boundary. Route geometry must reach prediction time as route geometry; a left/straight/right command remains useful context but cannot replace a local polyline. The policy-facing route must follow the ego travel lane rather than an unrelated road center. Ego history, speed, acceleration, camera availability, static hazards, and dynamic actors must retain consistent geometry and motion fields where those fields are available. Visibility diagnostics must not fabricate obstacles, and command-proxy route fallbacks must be labeled diagnostic rather than claim-valid.

B. Temporal Contract

The temporal contract aligns policy outputs with the runtime clock. Source and target horizons are explicit, timestamps are monotonic, and the current ego origin anchors interpolation. Matching grids preserve the trajectory, while mismatched grids are resampled deterministically and headings are recomputed on runtime points. Non-finite, malformed, or structurally invalid trajectories must be rejected or handled safely.

C. Lifecycle Contract

The lifecycle contract turns a policy into a persistent service. Session creation, active state, close, cleanup, and duplicate close are explicit. Late image, egomotion, route, and close events are counted and traced instead of crashing the service. Session state must not leak across repeated or interleaved sessions, and fatal errors must be distinguished from recoverable lifecycle warnings.

D. Deployment and Evidence Contracts

The deployment contract materializes the run: driver and wizard commands, topology, ports, scene identifiers, timeout, policy configuration, external-driver address, simulator Git state, patch state, Docker image identity, GPU state, and checkpoint hashes where applicable. The evidence contract defines when a rollout can support a paper claim: unique run identifiers, immutable manifests, terminal status, route-source and sensor-freshness validity, conformance checks, audits, support-bundle status, failed-run retention, and hashes tying generated numbers to artifacts.

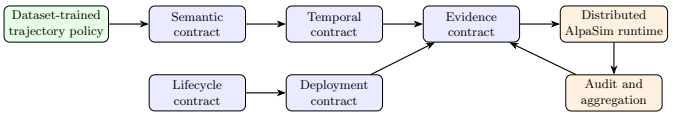

Fig. 1. WOD2Sim localizes integration failures at contract boundaries between a WOD-style policy and a distributed AlpaSim runtime.

III. WOD2SIM ARCHITECTURE

Figure 1 shows the system boundary. WOD2Sim sits between an offline or dataset-trained trajectory policy and the distributed AlpaSim runtime. The adapter reconstructs policy-facing state, preserves route geometry, resamples outputs, and isolates model discovery from unrelated optional backends. The runtime side records deployment state and evidence artifacts before any claim is accepted.

The implementation registers dependency-light baselines and optional gated adapter entry points through AlpaSim model discovery. Constant-velocity and route-following baselines are the public core because they require no learned checkpoint, no oracle actor proxy, and no bundled private artifact. Token BC/DAgger and direct actor-aware planning entry points are present for users with legitimate local checkpoints or scene-matched actor-proxy artifacts, but those paths are extension surfaces. The current contract-validation matrix (CVM) therefore treats missing direct-actor proxy state as an optional-extension deployment blocker, not as missing public core functionality or a policy failure.

IV. IMPLEMENTATION AND EVALUATION METHOD

The CVM package uses deterministic configuration files for core, semantic ablation, temporal ablation, lifecycle stress, and fault-injection matrices. The core matrix is configured for three policies and fifteen locally cached front-camera scenes. The current launcher does not expose a simulator seed override, so each policy/scene pair is counted as one closed-loop execution and the configured identifier is retained only as provenance metadata. The semantic ablation configures paired full-route and command-only route variants on fifteen locally cached scenes, while the temporal ablation remains a gated direct-actor resampling matrix. Lifecycle stress configures repeated service-harness cycles, and the fault matrix enumerates semantic, temporal, lifecycle, deployment/plugin-dependency, and evidence injections.

Scene selection is intentionally conservative. 15 scenes are selected from available local 26.02 front-camera USDZ artifacts, but the repository does not currently provide authoritative metadata proving straight, intersection, lane-change, dense-traffic, occlusion, or merge categories. The manifest therefore records 15 closed-loop scene(s) as available but unclassified. Each run manifest carries this `scenario_category`, asset

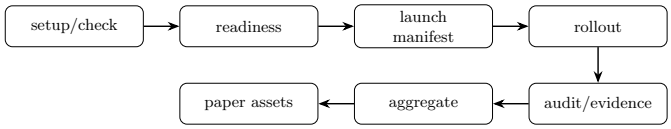

Fig. 2. The contract-validation pipeline separates setup, readiness, manifest materialization, rollout, audit, aggregation, and paper-asset generation.

availability, selection rationale, route/interaction feature expectations, and license-gating status; the paper does not claim scenario-category coverage.

The evaluation runner expands matrices into rows, writes CSV and JSON summaries, and returns nonzero status when any row is not completed. Aggregation preserves failed and blocked rows, separates planned rows from blockers, rejects duplicate completed run identifiers, writes LaTeX tables with data hashes, and generates vector figures. In the current environment, completed closed-loop rows are audited for route-source and sensor-freshness validity before they are counted as diagnostic rollout evidence. Public synthetic lifecycle and fault-injection harness rows are retained separately as service-level conformance diagnostics. The resulting rows document the exact experiment denominator while preventing unexecuted protocols from becoming results.

V. Results

The primary result is integration effectiveness, not policy ranking. WOD2Sim executes the dependency-light public core, audits valid full-contract rollouts without false blocking, and rejects route-invalid ablations before they can be treated as policy evidence. Table II reports the generated core policy status and the closed-loop diagnostic metrics currently available for dependency-light policies.

The configured matrices contain 148 rows: 45 mixed core-configuration rows, 30 semantic-ablation rows, 18 temporal-ablation rows, 40 lifecycle rows, and 15 fault-injection rows. The current aggregate attempts 115 rows and completes 115 rows, including 60 closed-loop rollout(s). Among completed full-contract rollouts, 42/ 45 pass the route and sensor audit. The public core itself contains 30 dependency-light row(s); 30/30 complete, 28 pass the audit, and 0 are blocked. Optional gated extension rows remain visible separately: 33/33 are blocked, including 33/33 direct actor-aware rows. The false-block check currently observes 0/ 42 valid full-contract rows incorrectly blocked by the evidence gate. Matched semantic-confound evidence includes 15/15 metric-bearing pairs. Across those pairs, full-contract minus command-only route means are -0.049 progress, -0.021 relative progress, 0.067 collision-any, 0.000 off-road, and 0.016 plan deviation. These deltas show that losing route geometry changes measured rollout behavior, but they are not a policy-quality comparison. The aggregate also keeps three completed full-contract rows from the same scene outside policy attribution because their audits found late command-proxy route fallback despite successful rollout completion; that is an integration-boundary finding, not a policy failure. The semantic rows are therefore an effectiveness

TABLE II
Public core dependency-light policy matrix status. Rows are constant-velocity and route-following rows only; optional gated direct-actor and learned-checkpoint extensions are excluded from this table and reported as blockers.

Public core policy	Rows	Done/att.	Audit	Progress	Coll./off	Lat. p95	Crash	Blocked
constant_velocity	15	15/15	14/15	0.525	0.667/0.000	–	0	0
route_following	15	15/15	14/15	0.477	0.733/0.000	–	0	0

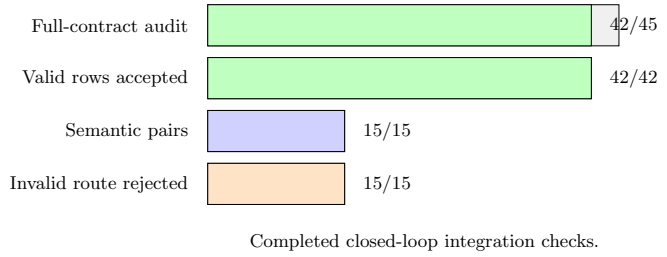

Fig. 3. Completed closed-loop integration checks. Bars report audit-valid full-contract rows, accepted valid rows, matched semantic metric pairs, and rejected command-proxy route rows; unavailable rows remain reported as blockers in the aggregate.

comparison at the semantic integration boundary: a functional command-only wrapper completes 15/15 metric-bearing rows that a non-contract path could incorrectly accept as policy evidence, whereas WOD2Sim rejects 15/15 of those invalid route rows and improves attribution on 15 rows. The aggregate keeps 0 rows as planned/not launched and 33 rows as true precondition or unsupported-ablation blockers.

The scene-coverage gate reports 15 unique closed-loop scene identifier(s), but only 0/ 6 required scenario categories are verified by authoritative metadata, and 15 closed-loop scene(s) remain unclassified. These rows are integration instances for exercising the contract boundary, not evidence of autonomous-driving scenario-category coverage.

Table III is the primary effectiveness evidence because it measures what the contracts buy on executed closed-loop rows. Completed full-contract rollouts test whether a valid integration survives the audit without false blocking, matched semantic-ablation rows expose a real route-loss confound in closed-loop runs, and the command-only baseline tests whether a runnable non-contract wrapper would leave invalid route metrics in the evidence stream. The contract gate changes that outcome by rejecting the same route-invalid rows as non-claim-valid. These checks answer the integration question directly for the semantic boundary: the contracts make runnable integrations auditable, and they keep boundary violations out of policy-attribution claims. The generated synthetic lifecycle and fault-localization artifacts remain service-level conformance diagnostics only; they are not used as evidence that WOD2Sim is faster, safer, or more correct than a functional non-contract wrapper in real simulator stress trials.

The same aggregate also reports failure attribution. It contains 42 contract-valid closed-loop row(s), 18 completed closed-loop row(s) rejected as integration/evidence-invalid, 33 precondition-blocked row(s), 33 integration-failure-attributable row(s), 55

TABLE III
CLOSED-LOOP INTEGRATION-EFFECTIVENESS CHECKS GENERATED FROM RETAINED ROLLOUT AND AUDIT ARTIFACTS. SYNTHETIC LIFECYCLE AND FAULT ROWS ARE EXCLUDED FROM THIS TABLE.

Closed-loop integration check	Observed	Denom.
Full-contract audit-valid rollouts	42	45
Valid full-contract rows accepted	42	42
Semantic ablation metric pairs	15	15
Naive command-only route rows with metrics	15	15
Naive invalid route evidence accepted	15	15
Contract invalid route evidence rejected	15	15
Invalid-row attribution improvement	15	15
Policy-behavior diagnostic rows	42	60
Verified scenario categories	0	6

synthetic diagnostic row(s), and 0 claim-valid policy benchmark row(s). These counts are the operational separation between integration failure and policy failure in the current release: 42 audit-valid full-contract closed-loop row(s) are policy-behavior-attributable diagnostic evidence, 0 row(s) are policy-failure-attributable, and 106 row(s) remain non-policy-attributed. No blocked, contract-invalid, synthetic, or benchmark-incomplete row can assign policy failure, even when its behavior metrics are degraded. A policy failure also requires the retained failure layer to be policy, not merely a degraded behavior metric.

VI. DISCUSSION, LIMITATIONS, AND RELATED WORK

The framework generalizes beyond a single simulator because the contracts identify boundary assumptions rather than AlpaSim-specific filenames. The five contracts are an engineering instantiation of contract-based cyber-physical system design and interface-level reasoning [1], [2]: an offline policy assumes route, timing, lifecycle, deployment, and evidence conditions, while the simulator boundary must either guarantee them or reject the run as non-claim-valid. Route geometry, sampling cadence, lifecycle robustness, deployment provenance, and claim-valid evidence also matter in WOD-style datasets [3] and in closed-loop planners and simulators such as nuPlan [4], CARLA [5], Waymax [6], and NAVSIM [7]. Scenario and falsification tools such as Scenic, VerifAI, DriveFuzz, and PlanFuzz [8], [9], [10], [11] exercise autonomous-driving systems with generated or mutated conditions; WOD2Sim is complementary because it gates whether the simulator/policy boundary itself preserved the assumptions needed to interpret a rollout. WOD2Sim uses AlpaSim as the case study because its external-driver boundary makes these contracts explicit.

To test whether the contract layer remains portable beyond the repository's local evaluation scripts, WOD2Sim also includes an AlpaSim E2E-style external-driver compatibility harness. This path is not reported as a challenge submission or leaderboard result. It is an external integration-validation target: the evaluator owns the simulator stack, scenes, and metrics, while WOD2Sim records whether route, sensor, temporal, lifecycle, deployment, and evidence assumptions still hold behind that managed boundary. A successful run would support interface portability; a failed run would localize the breakage to a contract layer before any returned metric is treated as policy behavior.

The limitations are material. WOD2Sim is not an official Waymo challenge interface, does not reconstruct complete WOD worlds, and does not redistribute restricted scene assets. The current CVM package has no completed claim-valid benchmark matrix, no validated scene-category coverage (0/ 6 required categories verified), no public learned checkpoint, no bundled redistributable restricted-scene subset, no scene-matched direct-actor proxy for actor-aware planning, and no simulator-backed lifecycle or fault-injection stress trial. These absences mean the public repository is not a complete benchmark release. They block learned-policy, direct-actor, restricted-asset, scenario-coverage, and benchmark claims. What is complete is narrower: the 30/30 dependency-light public core and the executed semantic route-boundary ablation. Their role is to show that the public contract layer runs, audits, and separates integration-valid from integration-invalid rows without mislabeling the missing benchmark prerequisites as policy failures. Lifecycle and fault-injection evidence still comes from public synthetic harnesses, so those counts are conformance checks rather than simulator-backed evaluation metrics.

These limitations also explain why the paper does not compare policy quality. A framework paper can report reliability, validity, and diagnostic coverage only after the corresponding rows are executed and audited. The supported success claim is narrower: 30/30 public-core rows executed, 42 full-contract closed-loop rows passed the route/sensor audit and were not falsely blocked by the evidence gate, and 15 runnable command-only route rows produced metrics that a naive wrapper could accept, while 15/ 15 were rejected as non-claim-valid route evidence by the contract gate. The 33 blocked optional-extension rows remain visible as remaining work, not as the primary result.

VII. FUTURE WORK

Future work will complete the optional benchmark path rather than weakening the current integration claim. The next steps are to generate scene-matched direct-actor oracle proxies, add a redistributable scene subset with authoritative scenario categories, package legitimate learned checkpoints, execute the temporal and direct-actor matrices, run the external-evaluator compatibility harness against a local or official challenge-style evaluation, retain the returned metrics with provenance, and compare time-to-diagnosis against a functional non-contract wrapper. Those additions would support learned-policy, scenario-coverage, portability, and benchmark claims only after their rows pass the same contract and evidence gates used here.

VIII. CONCLUSION

Integrating a dataset-trained trajectory policy into distributed closed-loop simulation requires more than input/output conversion. WOD2Sim formulates the boundary as five contracts: semantic, temporal, lifecycle, deployment, and evidence. The

current repository shows that this contract layer is already operational on the dependency-light public core: valid rows execute, route-loss ablations measurably alter behavior, invalid route evidence is rejected, and missing prerequisites remain visible as blockers instead of being blurred into policy outcomes. The paper still does not claim a full public benchmark, scenario-category coverage, or learned-policy result. Its stronger and more defensible conclusion is narrower: contract-based integration can make failure attribution trustworthy before policy-quality conclusions are drawn. That separation is the release-critical result because it prevents route, timing, lifecycle, deployment, evidence, and missing-prerequisite failures from being misreported as policy failures until the complete audited matrices exist.

APPENDIX
WOD2SIM REPOSITORY EVIDENCE MAP

The repository and paper share one release surface. The canonical manuscript is the root-level PDF. The paper source, bibliography, generated tables, figure inputs, matrix configuration, row manifests, aggregate CSV and JSON files, and release reports are all kept in named repository directories and rebuilt by the top-level release targets. Obsolete manuscript sources were removed from the release tree so the root PDF and `paper/cvm` source form the single paper surface.

Repository-native code provides the implementation evidence. Simulator adapters implement the semantic and temporal contracts; CLI commands materialize launch, readiness, audit, support-bundle, and reproduction steps; aggregation regenerates every number, table, and figure used in the build from retained matrix evidence.

This appendix is a traceability map, not an additional result. The current root PDF, README, reports, and generated evidence all preserve the same claim boundary: synthetic diagnostics are public service-harness evidence, while closed-loop scene rows are not claim-valid until executed, audited, and retained in the aggregate denominators.

The public repository intentionally separates source, execution evidence, and paper assets. The implementation layer contains the simulator adapters, route and scene-signal conversion, temporal resampling, plugin entry points, setup/readiness commands, audit logic, and support-bundle generation. The evaluation layer contains the CVM configuration, scene manifest, row manifests, retained CSV/JSON summaries, blocked-run reports, and generated table and figure assets. The manuscript layer consumes only those generated artifacts; manual edits to tables are not part of the release workflow.

The temporal adapter is traceable at unit-test level. Runtime frequency and horizon are validated as positive finite values; trajectories must be finite N-by-2 arrays; the current ego pose is the interpolation origin; optional source timestamps must be finite, strictly increasing, and inside the configured horizon; and headings are recomputed on the resampled runtime grid. These tests support the temporal contract statement, but they do not substitute for the blocked temporal-ablation scene matrix.

The three paper figures serve different audit roles. The architecture figure identifies where semantic, temporal, lifecycle, deployment, and evidence failures cross the external-driver boundary. The evaluation-pipeline figure shows how readiness, launch manifests, rollout status, audit, aggregation, and paper assets are chained. The results figure shows completed closed-loop integration checks; configured, planned, and blocked denominators remain auditable in the generated tables and reports rather than being presented as successes.

The release package also records what is absent. Gated scenes, private credentials, learned checkpoints, and proprietary container layers are not bundled. When they are needed for a future rollout, the repository records identifiers, hashes, commands, and precondition states so that restricted material does not become part of the public release and missing inputs are not misreported as policy behavior.

The intended reading order for the public release is therefore not only the paper. The repository inventory records the source tree, environment, build commands, and known gated prerequisites. The contract-test audit maps each required semantic, temporal, lifecycle, deployment/plugin-dependency, evidence, and fault-injection behavior to executable tests or to an explicit gap. The claim-evidence matrix is the final gate: a statement may appear as a paper claim only when it names the producing command, test, and evidence that support it. The blockers report keeps unavailable closed-loop rows visible, while planned rows remain visible without being misclassified as infrastructure blockers.

This structure is deliberately stricter than a demo-first release. The synthetic demo proves that command materialization, audit generation, support-bundle construction, and public plotting work without restricted assets. It does not prove closed-loop policy performance. Conversely, the matrix runner can create manifests for closed-loop scene rows before launch, preserving scene identifiers, policy names, execution identifiers, scenario categories, configuration hashes, runtime seed metadata when available, and precondition states. That makes a future rerun additive: once assets, actor proxies, ablation flags, and runtime dependencies exist, new completed rows can be aggregated beside the retained planned and blocked rows rather than replacing them.

The release commands follow the same separation. Inventory captures repository and environment state; check runs lint, conformance, and submission validation; demo generates a public synthetic run; synthetic evaluation executes lifecycle and fault-injection harnesses; scene evaluation records completed, planned, and blocked rows; aggregation regenerates result tables and figures; the paper target rebuilds the sole root-level PDF; and validation checks page count, A4 size, metadata, generated-artifact hashes, unresolved references, forbidden placeholder text, private paths, credentials, and duplicate manuscript PDFs. These gates are engineering controls, not scientific results.

REFERENCES

[1] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, "Taming dr. frankenstein: Contract-based design for cyber-physical systems," *European Journal of Control*, vol. 18, no. 3, pp. 217–238, 2012.

[2] L. de Alfaro and T. A. Henzinger, "Interface automata," in *Proceedings of the 8th European Software Engineering Conference*, pp. 109–120, ACM, 2001.

[3] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2446–2454, 2020.

[4] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, "nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles," in *CVPR Workshop on Autonomous Driving*, 2021.

[5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.

[6] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mougin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Advances in Neural Information Processing Systems*, 2023.

[7] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems*, 2024.

[8] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, "Scenic: A language for scenario specification and scene generation," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 63–78, ACM, 2019.

[9] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, "VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems," in *Computer Aided Verification*, pp. 432–442, Springer, 2019.

[10] S. Kim, M. Liu, J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2022.

[11] Z. Wan, J. Shen, J. Chuang, X. Xia, J. Garcia, J. Ma, and Q. A. Chen, "Too afraid to drive: Systematic discovery of semantic DoS vulnerability in autonomous driving planning under physical-world attacks," in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2022.